UNIVERSITÉ FRANÇOIS-RABELAIS DE TOURS

ÉCOLE DOCTORALE : SANTÉ, SCIENCE, ET TECHNOLOGIES
Laboratoire d'Informatique

THÈSE présentée par :

Nguyen HUYNH TUONG

soutenue le : 17 juin 2009

pour obtenir le grade de : Docteur de l'Université François-Rabelais de Tours

Discipline/Spécialité : INFORMATIQUE

COMPLEXITÉ ET ALGORITHMES POUR L'ORDONNANCEMENT MULTICRITERE DE TRAVAUX INDÉPENDANTS : PROBLÈMES JUSTE-À-TEMPS ET TRAVAUX INTERFÉRANTS

THÈSE dirigée par :

SOUKHAL Ameur	Maître de conférences, Université François Rabelais de Tours
BILLAUT Jean-Charles	Professeur, Université François Rabelais de Tours

RAPPORTEURS :

BAPTISTE Philippe	Chargé de Recherche CNRS, HDR (Paris)
CHRÉTIENNE Philippe	Professeur, Université Pierre et Marie Curie (Paris VI)

JURY :

AGNETIS Alessandro	Professeur, Université de Sienne, Italie
BAPTISTE Philippe	Chargé de Recherche CNRS, HDR (Paris)
BILLAUT Jean-Charles	Professeur, Université François Rabelais de Tours
CARLIER Jacques	Professeur, Université de Technologie de Compiègne
CHRÉTIENNE Philippe	Professeur, Université Pierre et Marie Curie (Paris VI)
NERON Emmanuel	Professeur, Université François Rabelais de Tours
SOUKHAL Ameur	Maître de conférences, Université François Rabelais de Tours

UNIVERSITÉ DE TOURS

ATTESTATION DE REUSSITE AU DIPLOME

Le Secrétaire Général atteste que

le Diplôme de docteur d'Informatique
a été décerné à

Monsieur HUYNH TUONG NGUYEN
né le 28 décembre 1978 à HO CHI MINH VILLE (VIETNAM)

au titre de l'année universitaire 2008/2009 avec la mention Très Honorable avec Félicitations

Titre des travaux	: Complexité et algorithmes pour l'ordonnancement multicritère des travaux indépendants : Problèmes juste-à-temps et travaux interférants.
Date de soutenance	: 17 juin 2009
Etablissement soutenance	: UNIVERSITE DE TOURS
Jury	: M. JACQUES CARLIER, President du jury, PROFESSEUR DES UNIVERSITES Université de Technologie de COMPIEGNE M. ALESSANDRO AGNETIS, PROFESSEUR UNIVERSITA DEGLI STUDI DI SIENA (ITALY) M. PHILIPPE BAPTISTE, CHARGE(E) DE RECHERCHE HDR CNRS M. JEAN CHARLES BILLAUT, PROFESSEUR DES UNIVERSITES UNIVERSITE DE TOURS M. PHILIPPE CHRETIENNE, PROFESSEUR DES UNIVERSITES UNIVERSITE PARIS 6 M. EMMANUEL NERON, PROFESSEUR DES UNIVERSITES UNIVERSITE DE TOURS M. AMEUR SOUKHAL, MAITRE DE CONFERENCES UNIVERSITE DE TOURS
Ecole doctorale	: Santé, Sciences,Technologies
Formation doctorale	: S 20 007036 - Informatique

Fait à TOURS, le 24 juin 2009

Le Secrétaire Général

Pierre GABETTE

N° étudiant : 20404091

Avis important: Il ne peut être délivré qu'un seul exemplaire de cette attestation. Aucun duplicata ne sera fourni.

À ma famille : mes parents, ma femme et ma fille

Remerciements

Les travaux réalisés au cours de cette thèse ont été effectués au sein du Laboratoire d'Informatique de l'Université de François Rabelais de Tours (EA 2101), dans l'équipe Ordonnancement et Conduite.

Je souhaite tout en premier lieu remercier Ameur Soukhal, Maître de Conférences à l'Ecole Polytech'Tours, qui a encadré cette thèse. Pendant ces trois années, il a su orienter aux bons moments mes travaux de recherches en me faisant découvrir l'ordonnancement au travers de son regard novateur et critique. Ses conseils et ses commentaires précieux m'ont permis de surmonter les difficulté et de progresser.

Je tiens à exprimer mes remerciements à Jean-Charles Billaut, Professeur de l'Ecole Polytech'Tours et également mon directeur de thèse, pour ses encouragements, ses conseils et sa confiance.

J'adresse tous mes sincères remerciements à Philippe Chrétienne, Professeur de l'Université Pierre et Marie Curie (Paris VI), et à Philippe Baptiste, Professeur de l'École Polytechnique (LIX), qui m'ont fait l'honneur d'accepter d'être rapporteurs de mes travaux.

Mes chaleureux remerciements s'adressent à Jacques Carlier, Professeur de l'Université de Technologie de Compiègne d'avoir accepté d'être examinateur et président du jury.

Je remercie également les autres membres du jury qui ont accepté de juger ce travail : Alessandro Agnetis, Professeur de l'Université de Siena (Italy) et Emmanuel Néron, Professeur de l'Université François Rabelais de Tours.

Je remercie le Ministère de l'Éducation et la Recherche pour le financement qui m'a été accordé pour le bon déroulement de ma thèse.

Plus largement, je voudrais remercier les différentes personnes du Laboratoire d'Informatique et du Département Informatique de Polytech'Tours auprès desquelles je suis souvent venue chercher conseil et avec lesquelles j'ai partagé de très bons moments, tant pour le travail que pour des instants de détente. Grâce à un partage et une bonne convivialité entre les membres de l'équipes, j'ai eu l'opportunité d'effectuer ma thèse dans d'excellente condition. Parmi ceux qui ont contribué à mon travail, je remercie tout spécialement Jean-Louis Bouquard et Vincent T'Kindt pour les conseils scientifiques et leur supports. Dans ces remerciements je n'oublie jamais les autres membres de l'équipe qui ont fait de mon séjour, une période très agréable et enrichissante. Je pense en particulier à Christian Proust (Directeur de l'Ecole Polytech'Tours), Patrick Martineau, Christophe Lenté, Claudine Tacquard, Carl Esswein, Geoffey Vilcot, Cédric Pessan, Mathieu Pérotin, Cédric Mocquillon, Mathieu Rouleau, Yanick Kergosen, Gaël Sauvanet, et Rabah Belaid. Mes remerciements sont adressés également aux étudiants du cycle d'ingénieur qui sont intervenus dans mes projets de recherche : Brien Lit-

teaut, Paul Vignard, Corentin Del'homme, Dan Shao, Zangou Dao, Laurent Miscopein et Daudé Guillaume.

Enfin, mes remerciements vont à ma femme My-Dung ainsi qu'à nôtre petite fille Gia-An, qui sont à mes côtés depuis le début et qui m'ont toujours écoutées et accompagnées dans cette aventure avec beaucoup de patience et surtout d'amour.

Résumé

Nous abordons dans cette thèse des problèmes d'ordonnancement de travaux indépendants sur une machine ou sur des machines parallèles. Plus précisément, nous abordons deux catégories de problèmes :

1. les problèmes d'ordonnancement de type juste-à-temps : il s'agit de déterminer un ordonnancement de sorte que les travaux se terminent le plus près possible de leur date de fin souhaitée. On considère le cas où la date de fin souhaitée commune est connue et le cas où elle est à déterminer. De nouveaux algorithmes exacts sont proposés - gloutons et programmes dynamiques -. Des schémas d'approximation sont élaborés.
2. les problèmes d'ordonnancements de travaux interférants : il s'agit de déterminer un ordonnancement qui permet d'optimiser un critère pour la globalité des travaux à effectuer, sachant que la solution trouvée doit permettre également l'optimisation d'un autre critère défini uniquement sur un sous-ensemble des travaux. Il s'agit ici d'un nouveau problème d'ordonnancement multicritère, différent de la notion classique, et qui se rapproche des problèmes de type "*multi-agent scheduling*" ou "*interfering job sets*". Les approches considérées pour trouver une solution non dominée sont l'approche ε-contrainte, la combinaison linéaire de critères et le *goal programming*. De nouveaux résultats de complexité sont montrés et des algorithmes polynomiaux/pseudo-polynomiaux sont développés pour le calcul de cette solution non dominée.

Mots-clés : ordonnancement, une machine, machines parallèles, juste-à-temps, travaux interférants, complexité, programmation dynamique, schéma d'approximation

Abstract

In this thesis we consider scheduling problems of independent jobs on a single machine or on parallel machines. More precisely, we tackle two kinds of problems :

1. just-in-time scheduling problems : it aims to determine a schedule so that a job completes as close as possible to its due date. We consider the case where the common due date is known and the case where the common due date has to be fixed. New exact algorithms based on greedy algorithms and dynamic programming are proposed. Approximation schemes are given.
2. scheduling problems with interfering jobs : the aim is to determine a schedule that optimizes a criterion for the whole set of jobs and so that the solution optimizes another objective only for a subset of jobs. It is here a new multi-criteria scheduling problem, different from the classical notion, which is related to "multi-agent" or to "interfering job sets" scheduling problems. The approaches considered for finding a non-dominated solution are the ε-constraint approach, the linear combination of criteria and the goal programming approach. New complexity results are proposed and polynomial/pseudo-polynomial algorithms are developed for the calculation of the non-dominated solution.

Keywords : scheduling, single machine, parallel machines, just-in-time, interfering jobs, complexity, dynamic programming, approximation scheme

Table des matières

Introduction générale **1**

I Ordonnancement juste-à-temps à date due commune **5**

1 Introduction à l'ordonnancement juste-à-temps **7**
1.1 Systèmes "Juste-à-temps" 7
1.2 Mesurer l'avance/retard d'un travail 10
1.3 Mesurer le coût total d'avances et retards des travaux 12
1.3.1 Fonction de coût linéaire continue 13
1.4 Dates de fin souhaitée 14
1.4.1 Date de fin commune donnée 15
1.4.2 Famille de dates de fin commune donnée 16
1.4.3 Dates de fin commune contrôlable 16
1.4.4 Dates de fin souhaitées généralisées 17
1.5 Complexité et approximation polynomiale 19
1.5.1 Approximation et garanties de performances 19
1.5.2 Schémas d'approximation polynomiaux 21
1.6 Problèmes d'ordonnancement Juste-à-temps abordés 24

2 Retard pondéré avec une date de fin donnée **25**
2.1 Introduction 25
2.2 État de l'art 27
2.2.1 Approximabilité du retard pondéré 30
2.2.2 Remarques sur programme dynamique de Lawler et Moore . . . 31
2.3 Problème d'ordonnancement à une seule machine 34
2.3.1 Nouveau programme dynamique 34
2.4 Problème d'ordonnancement à machines identiques 39
2.4.1 Programme dynamique pour le cas de machines identiques . . . 39
2.4.2 Extension au cas de machines uniformes 43
2.5 Approximabilité 46

3 Avance et retard pondérés avec dates de fin données **49**
3.1 Introduction 50
3.2 Propriétés pour le cas d'une date de fin souhaitée commune 53
3.3 Durées opératoires identiques 55
3.3.1 Une seule machine avec une date de fin souhaitée commune . . . 56

3.3.2 Une seule machine avec deux dates de fin souhaitées 61
3.3.3 Une seule machine avec famille de dates de fin souhaitées 66
3.3.4 Machines uniformes avec famille de dates de fin souhaitées . . . 69
3.4 Durées opératoires quelconques 70
3.4.1 PTAS pour le cas d'une seule machine 70
3.4.2 PTAS pour le cas de m machines parallèles identiques 87

4 Avance et retard pondérés avec dates de fin contrôlables 91
4.1 Introduction 92
4.1.1 Quelques notations spécifiques à ce chapitre 95
4.2 Durées opératoires identiques 97
4.2.1 Une seule machine avec une seule date de fin souhaitée 97
4.2.2 Machines identiques avec famille de dates de fin souhaitées . . . 103
4.2.3 Machines uniformes avec une seule date de fin souhaitée 110
4.3 Durées opératoires quelconques 118
4.3.1 Une seule machine avec une seule date de fin souhaitée 118
4.3.2 Machines parallèles avec une seule date de fin souhaitée 121
4.4 Conclusion 127

5 Conclusion et perspectives de la première partie 129

II Ordonnancement de travaux interférant indépendants 133

6 Introduction à l'ordonnancement des travaux interférants 135
6.1 Motivation et contexte de travail 135
6.2 Brève introduction de l'ordonnancement multi-critère 137
6.2.1 Généralités 137
6.2.2 Classes de méthodes de résolution 141
6.3 Définition de l'ordonnancement avec des travaux interférants 142
6.4 État de l'art 143
6.5 Intérêt de l'étude 146

7 Problème d'ordonnancement à une seule machine 149
7.1 Introduction 149
7.2 Problèmes résolus en temps polynomial 151
7.3 Problèmes NP-difficiles au sens ordinaire 153
7.4 Problèmes NP-difficiles au sens fort 161
7.5 Problèmes ouverts 169

8 Problème d'ordonnancement à machines parallèles 171
8.1 Introduction 171
8.2 Résultats de complexité 172
8.3 Problèmes ouverts 173
8.4 Algorithme de programmation dynamique 173
8.4.1 Formulation générale de programmation dynamique 173
8.4.2 Une application de la formulation générale 174
8.4.3 Problèmes avec les fonction objectifs $\sum C_j(\mathcal{N}_1)$ et $\sum C_j(\mathcal{N})$ 175
8.4.4 Problèmes avec les fonction objectifs $\sum w_j C_j(\mathcal{N}_1)$ and $C_{\max}(\mathcal{N})$. 177
8.4.5 Problèmes avec les fonction objectifs $\sum w_j C_j(\mathcal{N})$ and $C_{\max}(\mathcal{N}_1)$. 178
8.4.6 Problèmes avec les fonction objectifs $C_{\max}(\mathcal{N})$ et $C_{\max}(\mathcal{N}_1)$ 179

8.5 Conclusion 180

9 Conclusion et perspectives de la seconde partie **181**

Conclusion générale et perspectives **183**

Liste des tableaux

1.1 Classification des problèmes d'ordonnancement à affecter date de fin [126] 18
1.2 Tableau de problèmes abordés 24

2.1 État de l'art sur la minimisation des retards 29
2.2 État de l'art sur la minimisation des retards 31
2.3 Durées opératoires, dates de fin souhaitées, et pénalités 38

3.1 État de l'art sur la minimisation des avances et des retards 54
3.2 Durées opératoires, dates de fin souhaitées, et pénalités d'avance/retard 59
3.3 Durées opératoires et les poids (pénalités d'avance et de retard) des travaux 73

4.1 Problèmes JàT avec affectation de la date de fin souhaitée 95
4.2 Performance de la borne inférieure par rapport à une borne supérieure . 127

5.1 Bilan et perspectives 131

6.1 Résultats de complexité de l'ordonnancement multi-agent [121] 145

7.1 Ordonnancement avec travaux interférants sur une seule machine [118] 150

8.1 Ordonnancement avec travaux interférants sur machines parallèles . . . 172

Table des figures

1.1 Calcul de l'avance en fonction de date de fin souhaitée 11
1.2 Calcul de l'avance en fonction de la promptitude 11
1.3 Calcul de l'avance en fonction de date de début souhaitée 12
1.4 Calcul du retard 12
1.5 Fonction de coût d'avance/retard linéaire et continue 13
1.6 Juste-à-temps : critère non-régulièr 14
1.7 Classes d'approximabilité (sous l'hypothèse $\mathcal{P} \neq \mathcal{NP}$) [171] 23

2.1 Cas d'une seule machine - position du nouveau travail en retard 33
2.2 Cas d'une seule machine - nouvelle fonction de récurrence 36
2.3 Cas d'une seule machine - travail J_i est en avance 36
2.4 Cas d'une seule machine - travail J_i est en retard 36
2.5 Algorithme WTSMCDD 38
2.6 Fonction récurrente du cas de machines parallèles 40
2.7 Cas 1 - J_i est en avance sur M_1 41
2.8 Cas 2 - J_i est en retard sur M_1 41
2.9 Cas 3 - J_i est en avance sur M_2 42
2.10 Cas 4 - J_i est en retard sur M_2 42
2.11 Algorithme WTP2CDD 44
2.12 Algorithme WTQ2CDD 47

3.1 Positions des travaux sur une machine avec une date due donnée 57
3.2 Matrice des coûts d'affectation des travaux 59
3.3 Exemple avec 5 travaux 60
3.4 Matrice d'affectation avec 5 travaux 60
3.5 Matrice d'affectation avec 5 travaux et $(\delta_1, \delta_2) = (0, 3)$ 60
3.6 Solution optimale correspondant au Cas (a) 60
3.7 Matrice d'affectation avec 5 travaux et $(\delta_1, \delta_2) = (1, 2)$ 61
3.8 Solution optimale selon le cas (b) 61
3.9 Exemple avec temps morts dans $[D_0, D_1]$ et $]D_1, D_2]$ et après D_2. 62
3.10 Exemple sans temps mort dans $]D_1, D_2]$ 63
3.11 Exemple avec un temps mort dans $]D_1, D_2]$ 64
3.12 Exemple avec 5 travaux autour de D_1 et 4 travaux autour de D_2 66
3.13 Reformulation de la fonction objectif pour 9 travaux 72
3.14 Solution optimale S^* pour le $R2||\sum C_i$ 73
3.15 Une meilleure solution déduite de la solution optimale du $R2||\sum C_i$ . . . 73
3.16 Solution optimale du problème $1|d_j = d, non - restrictive|\sum(\alpha_i E_i + \beta_i T_i)$ 74

3.17 Solution déduite à partir de S' pour le $R2||\sum C_i$ 74
3.18 Exemple de remplacement des petits travaux 81

4.1 La matrice d'affectation des travaux modifiée 99
4.2 Durées opératoires et pénalités 99
4.3 Matrice des coûts d'affectation des 5 travaux 100
4.4 Solution optimale pour l'exemple choisi 100
4.5 Matrices d'affectations selon [165] 101
4.6 Deux travaux adjacents ordonnancés en avance 102
4.7 Différents cas de figures selon les premiers temps morts 104
4.8 Exemple avec un temps mort $X = 3p$ 107
4.9 Exemple avec un temps mort de durée $2p$ 108
4.10 Construction d'une solution optimale avec un temps mort de durée p . . 108
4.11 Exemple d'amélioration d'une solution avec $X = 2p$ par décalage à gauche108
4.12 Algorithme pour $Qm|p_i = p, d_i = np\lambda_m|\sum(\alpha E_i + \beta T_i)$ 111
4.13 Exemple d'application de l'algorithme AssQm1 avec 13 travaux 111
4.14 Courbes du coût d'avance et de retard d'un travail J_i 124

6.1 L'étude des problèmes d'ordonnancement multicritères [209] 140
6.2 Les méthodes de résolution de problèmes multicritères [209] 142

7.1 Une séquence initiale avec 10 travaux 157
7.2 Séquences S^0 et σ' et position des travaux $2i-1$ et $2i$ 159
7.3 Une séquence optimale σ du $1||F_\ell\big(\sum w_jC_j(\mathcal{N}), L_{\max}(\mathcal{N}_1)\big)$ 162
7.4 Une séquence optimale σ du $1||F_\ell\big(L_{\max}(\mathcal{N}), \sum w_jC_j(\mathcal{N}_1)\big)$ 164
7.5 Une séquence optimale σ du $1||\varepsilon\big(\sum w_jC_j(\mathcal{N}) / \sum w'_jC_j(\mathcal{N}_1)\big)$ 167

Introduction générale

Motivée et stimulée par des questions qui se posent dans la planification et la gestion de la production, la théorie de l'ordonnancement est devenue un domaine important d'optimisation combinatoire, situé à l'interface entre les mathématiques appliquées, l'informatique et la recherche opérationnelle. Dans son sens le plus large, l'ordonnancement consiste à programmer l'exécution d'une réalisation en attribuant des ressources aux tâches et en fixant leur date d'exécution. Pour plus de détails sur la théorie d'ordonnancement nous pouvons citer les livres de Carlier et Chrétienne [31], de Leung [155], de Pinedo [177], de Brucker [29], de T'Kindt et Billaut [209], de Blazewicz et al. [27] et le livre d'exercices du groupe de travail GOThA [79].

Depuis les années 50 du dernier siècle, les problèmes d'ordonnancement ont évolué et leurs études n'ont pas cessé d'augmenter, peut-être du fait de leur applicabilité à une multitude croissante de problèmes réels. De plus, parmi les problèmes d'optimisation combinatoire, les problèmes d'ordonnancement sont probablement ceux qui sont le plus liés au monde industriel, puisqu'il s'agit de concilier, de façon optimale, des ressources limitées avec des activités dans le temps. Ainsi, des domaines comme l'éducation, l'agriculture, le transport ou la santé ont bénéficié des recherches issues de l'ordonnancement. Grâce aux progrès énormes des ordinateurs, les problèmes d'ordonnancement approchent de plus en plus les problèmes réels, qui attirent les chercheurs par leur intérêt et leur difficulté [184].

On constate que *le client* prend une importance considérable dans les organisations et dans les systèmes de production en particulier. L'organisation des chaînes logistiques repose sur une coordination des acteurs allant des fournisseurs des fournisseurs jusqu'au client final. Les problèmes de synchronisation et de livraison nécessitent une coordination parfaitement réglée, qui peut être obtenue grâce à des études sur le juste-à-

temps (JàT) (*just-in-time* en anglais). D'autre part, la prise en compte des particularités des clients – certains sont plus exigeants sur les délais, certains sur les quantités, certains souhaitent grouper leurs livraisons, d'autres non, etc. – nécessite l'introduction de paramètres et de mesures dédiés à certains travaux et pas à d'autres. Par nature, les travaux sont en compétition pour l'utilisation des ressources, les travaux interfèrent pour leur utilisation et leurs objectifs ne sont pas nécessairement les mêmes. Ce genre de problème nécessite des études sur les travaux interférants.

Dans le cadre de l'étude théorique de cette thèse, nous nous focalisons sur des problèmes d'ordonnancement multicritères, dont l'étude a bien évolué dans les années récentes, et notamment sur deux approches très développées récemment. La première concerne l'approche multicrière de type 'Juste-à-temps' qui attire les chercheurs depuis les vingt dernières années par son intérêt pratique et sa difficulté. La seconde présente une type de problème d'ordonnancement multi-critère plus récent appelé "ordonnancement des travaux interférants".

Introduction à la partie I

Le critère lié à la minimisation simultanée des coûts d'avance et de retard fait l'objet de cette première partie de thèse. Ce critère est issu de la production juste-à-temps que nous présentons ci-dessous. La philosophie de la production JàT s'est développée depuis une soixantaine d'années comme une politique de production, qui permet de limiter les stocks et de fabriquer des produits qui correspondent exactement à la demande [32]. Sourd [199] indique que "*l'idée de base est d'éviter toute production anticipée de produits intermédiaires ou finaux, ce qui signifie que le producteur se contente de produire la quantité strictement nécessaire, au moment voulu, pour satisfaire la demande de chaque client. Ainsi, c'est la date à laquelle le client demande à être livré qui fixe la date de livraison du produit mais, le JàT va même plus loin : toutes les dates de toutes les opérations de la fabrication du produit livré sont également fixées de manière à ce qu'il n'y ait pas de temps mort dans la chaîne de fabrication. De cette manière, le produit est emballé juste-à-temps pour être livré, il est assemblé juste-à-temps pour être emballé et, ainsi de suite jusqu'à déterminer la date de production des matières premières*".

En effet, le concept du JàT a été préconçu au Japon au sein de l'usine Toyota par Taiichi Ohno et avait comme motivation principale l'élimination des gaspillages à tous les

niveaux [161]. L'introduction du système du Kanban a permis, au moyen d'étiquettes (Kanban signifie étiquette en japonais) de signaler les demandes de chaque poste de la chaîne de production, de mettre en oeuvre simplement une politique de production JàT : limitation de la production du poste amont aux besoins exacts du poste aval. Les articles de Huang et Kusiak [105] et Akturk et Erhun [8] offrent une synthèse sur la recherche récente dans ce domaine.

Dans cette partie, nous considérons des problèmes d'ordonnancement de travaux indépendants, sur une machine unique ou sur des machines parallèles. Nous nous intéressons plus particulièrement au cas où les travaux ont une date de fin souhaitée commune. Dans le chapitre 2, nous nous intéressons à la minimisation du retard pondéré total des travaux et proposons de nouveaux algorithmes de programmation dynamique. Dans le chapitre 3, nous tenons compte des pénalités d'avance, nous identifions de nouveaux cas polynomiaux dans le cas où les travaux ont des durées identiques et nous proposons de nouveaux schémas d'approximation dans le cas général. Dans le chapitre 4, nous considérons que la date de fin est commune aux travaux (ou à un sous-ensemble des travaux) et que cette date est à déterminer. De la même façon, nous identifions des cas polynomiaux et nous proposons quelques schémas d'approximation. Le chapitre 5 conclut cette partie.

Introduction à la partie II

La deuxième partie de cette thèse porte sur l'étude d'une nouvelle classe de problèmes d'ordonnancement multicritères. Dans la plupart des études en ordonnancement multicritère, tous les travaux contribuent de la même façon à la mesure de la qualité de la solution, et celle-ci peut être donnée par plusieurs critères. Par exemple, on considérera le maximum de toutes les dates de fin d'exécution des travaux comme un critère et le plus grand retard de tous les travaux comme un autre critère. Nous considérons dans cette partie qu'un critère est fourni par l'ensemble des travaux, mais que les autres critères peuvent porter sur un sous-ensemble des travaux. On voudra par exemple minimiser le maximum de toutes les dates de fin d'exécution des travaux, tout en respectant une borne pour le plus grand retard de certains travaux seulement. Ces problèmes se posent dans ces termes pour certaines entreprises où la production journalière est évaluée par un critère qui porte sur tous les travaux, et où des

sous-ensembles de travaux doivent respecter des contraintes – parfois imposées par les clients ou relatives aux retards déjà accumulés – selon d'autres critères.

Le chapitre 6 présente ces problèmes dans le contexte de la littérature et montre son originalité. Le chapitre 7 est consacré à l'étude des problèmes à une machine, des cas polynomiaux, NP-difficiles au sens ordinaire et NP-difficiles au sens fort sont identifiés. Le chapitre 8 porte sur l'étude des problèmes à machines parallèles. Le chapitre 9 conclut cette partie.

Une conclusion générale fait une synthèse des problèmes abordés, des résultats obtenus et des nombreuses perspectives de recherche qui découlent de ces études.

Première partie

Ordonnancement juste-à-temps à date due commune

Chapitre 1

Introduction à l'ordonnancement juste-à-temps

Dans ce chapitre, nous présentons d'une façon succincte le Juste-à-Temps (JàT) et ses objectifs ainsi que les différents niveaux de décisions qui apparaissent dans un système de production. Nous nous intéresserons par la suite aux problèmes d'ordonnancement JàT qui ont pour but de planifier l'exécution d'un travail dans un atelier de production. Après avoir présenté les différentes classes de problèmes rangés selon la fonction de coût ou selon la définition des dates de fin souhaitées des travaux, nous préciserons les problèmes d'ordonnancement JàT étudiés dans cette première partie de thèse. Quelques classes d'approximabilité et quelques définitions d'algorithmes à garantie de performance seront par la suite formellement exposées.

1.1 Systèmes "Juste-à-temps"

Les marchés sont de plus en plus exigeants sans pour autant renoncer à la diversité des produits ou autoriser la moindre augmentation des coûts. La pression économique de plus en plus forte et le niveau d'exigence croissant des clients obligent les entreprises

à réduire au maximum les stocks et les délais tout en essayant d'optimiser au mieux les coûts de production. La réduction des délais est un enjeu essentiel du processus de gestion d'une chaîne logistique. Il est donc nécessaire de se munir d'une véritable fonction capable de gérer la production en tenant compte des contraintes internes (sur l'exécution de l'opération actuelle et suivantes) et externes (comme les clients).

Dès les années 1920, lorsque la demande est devenue forte, l'industrie est passée de la production artisanale, à la production de masse. Tant que la question de la vente des produits ne se posait pas (par exemple les véhicules), le critère essentiel était la productivité. Puis, avec les crises des années 1970 il a fallu se tourner vers de nouveaux modes de production développés par les japonais (notamment par Taiichi Ohno dans les usines d'assemblage de voitures Toyota [161, 32]) pour leur marché national. Ainsi, on est passé de la production en flux poussés, à la production en flux tirés. Il s'agit donc d'un *système Juste-à-temps* (Jàt) qui peut être défini comme suit : *"C'est un système de gestion de la production en flux tendu visant la fabrication et le stockage des bonnes quantités au bon moment, à chaque étape du processus*". C'est la commande qui va déclencher la production et non plus la production qui déclenche la recherche d'un acheteur. Ainsi, la façon de gérer l'avancée des produits est renversée par rapport aux modèles pères tel que le modèle du lot économique (Harris) où la planification est basée sur une production annuelle régulière. Dans le modèle JàT, on parle de *flux tendus* pour une amélioration continue de la qualité et de la productivité et en essayant d'éliminer le *"Muda"* (*gaspillage* . Selon le président honoraire du C.A. Shoichiro Toyoda, Toyota *"le Gaspillage consiste en tout ce qui dépasse la quantité minimale requise en matériel, équipement, espace et temps pour ajouter de la valeur au produit*". Comme exemples de gaspillage, on peut citer :

- Surproduction et coûts de stockage
- Produits défectueux
- Méthodes inefficaces qui engendrent des attentes et des tâches inutiles
- Transports et manutentions inutiles

En effet, le gaspillage signifie non valeur ajoutée à détecter et éliminer pour accroître la productivité. La plus part des activités issues de *"Lean management"* sont orientées dans cette direction [158]. Comme le souligne Sensei Yoshida dans Lean Training Newsletter [225] *"être lean c'est arrêter de surproduire"*. En effet, la surproduction est le pire du gaspillages car elle engendre un surstock, une surqualité et un surdimensionnement des équipements, par conséquent, une réduction des profits des entreprises. Le concept

lean est donc plus large qu'une méthode de production et s'appuie principalement sur quatre niveaux d'analyse : une redéfinition de la valeur produite par une entreprise (en particulier, prendre en considération le point de vue du client pour la définition de la valeur ajoutée d'un travail), le développement d'un schéma productif caractéristique (tirer la production en fonction de la demande et non pas la pousser en fonction des capacités locales de production), le développement d'attitudes managériales originales (tout acteur du *système de production* doit penser continuellement à une amélioration du système productif) et la formulation d'une stratégie à long terme (viser l'excellence en permanence dans un contexte de concurrence). La pensé lean dépasse alors le cadre de l'organisation de la production puisqu'elle s'étend aux services administratifs, au développement de produit et au développement informatique (pour plus de détails, voir [220]

Au coeur de la pensée lean se repose le concept JàT. Le modèle JàT cherche alors à fournir au client la commande prévue, le jour prévu, au coût minimum. Cette approche permet d'assurer la satisfaction du client tout en garantissant la rentabilité des ressources, donc de réduire les coûts et d'optimiser les services en travaillant au plus près de la demande client en réduisant les temps d'attente et les stocks. Ainsi, on peut résumer les objectifs du modèle JàT comme suit :

- Produire au rythme de la demande
- Lots de petite taille pour réduire le cycle de production ainsi renverser la façon de gérer l'avancée des produits, en passant des flux poussés aux flux tendus
- Éliminer le gaspillage
- Amélioration continue
- Respect des délais

Une telle approche peut être déployée aux niveaux stratégique, tactique et opérationnel. En effet, les décisions à prendre au niveau stratégique définissent le long terme. Quant à celles du niveau tactique, i.e. en moyen terme, elles représentent principalement la planification de la production. Les décisions opérationnelles ont pour but de respecter les décisions prises aux niveaux stratégique et tactique tout en assurant le bon déroulement de l'atelier de production en termes de gestion de moyens de transformation (les machines, par exemple) et de transport, les stocks afin de satisfaire à temps les commandes de clients. Ainsi, le système JàT a des conséquences directes sur le plan de production à mettre en place pour le très court terme au sein de l'atelier. Les

modèles d'ordonnancement classiques basés sur des critères réguliers classiques (C_{max}, T_{max}, L_{max}, etc.) ne semblent donc pas toujours satisfaisants par rapport à une politique de production en flux tendus . La qualité d'un ordonnancement peut être ainsi mesurée en fonction de l'*avance/retard* du produit par rapport à sa date de livraison, autrement dit de sa date de fin souhaitée : un coût de stockage est généré pour chaque produit réalisé avant sa date de fin souhaitée ; s'il s'agit en plus d'un produit périssable, il se peut qu'il perde complètement sa valeur. De même, un coût de pénalité est généré si le produit est livré en retard, sans parler des risques de pertes de clients dûs à leur mécontentement. Chercher un ordonnancement JàT, c'est calculer les dates de début et de fin des traitements des opérations sur la(les) machine(s) tout en minimisant le coût total de l'avance et retard des travaux.

Selon les fonctions de coûts d'avance/retard à minimiser et selon les définitions des dates de fins souhaités à respecter, on se retrouve face à des problèmes d'ordonnancement JàT différents que nous allons souligner dans les sections suivantes.

1.2 Mesurer l'avance/retard d'un travail

Mesurer l'avance ou le retard d'un travail (ou d'une quantité de produits à livrer) peut se faire de plusieurs façons.

L'avance est généralement définie en fonction des dates de fin souhaitées. Pour un travail J_i, on note E_i son avance avec $E_i = \max(0, d_i - C_i)$ avec C_i la date de fin d'exécution réelle de J_i (voir Figure 1.1). C'est la définition retenue dans notre étude. Il est à noter que dans certains cas, l'avance d'un travail est calculée en fonction de sa date d'arrivée (date de début au plus tôt), notée r_i ; ou bien, par rapport à sa date de début d'exécution réelle notée s_i. Dans le premier cas, on parle de la *promptitude* où l'avance est définie par $E_i^r = \max(0, r_i - s_i)$, illustré par la Figure 1.2 (cf. [193, 144, 97, 215]). Le second cas a été traité principalement dans [90, 62]. Les auteurs considèrent le cas d'atelier en ligne (flow shop) où tous les travaux doivent être exécutés sur toutes les machines dans le même ordre. Dans leur étude, le coût d'avance d'un travail J_i est fonction de la date de début de traitement de la première opération de J_i sur la première machine ($s_{(1,i)}$), donnée par $E_i^s = \max(0, d_i^s - s_{(1,i)})$ avec d_i^s une date de début de traitement souhaitée de la première opération de J_i (voir Figure 1.3).

Le retard d'un travail J_i, noté T_i, est toujours défini en fonction de la date de fin

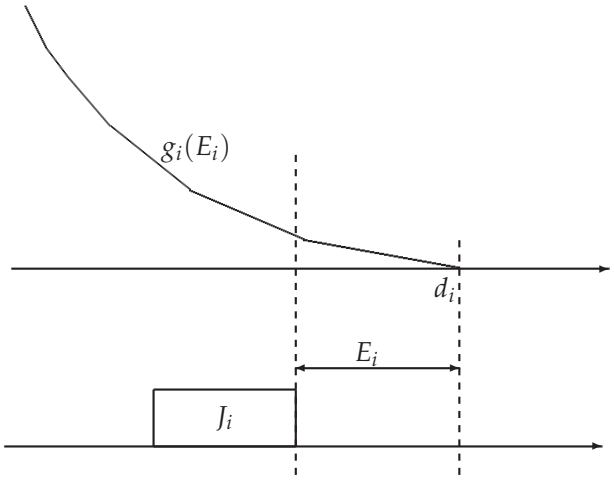

FIG. 1.1 – Calcul de l'avance en fonction de date de fin souhaitée

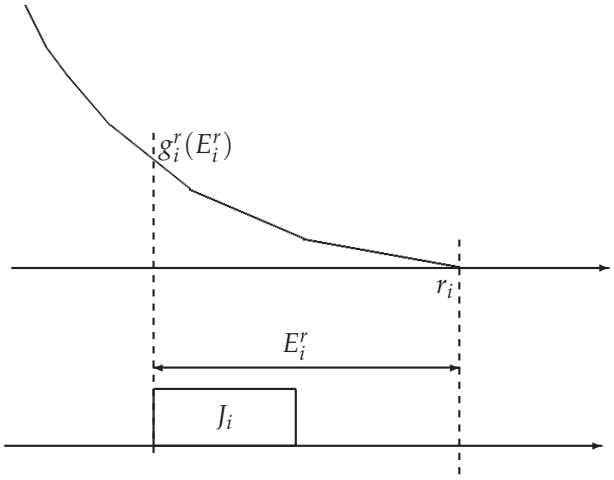

FIG. 1.2 – Calcul de l'avance en fonction de la promptitude

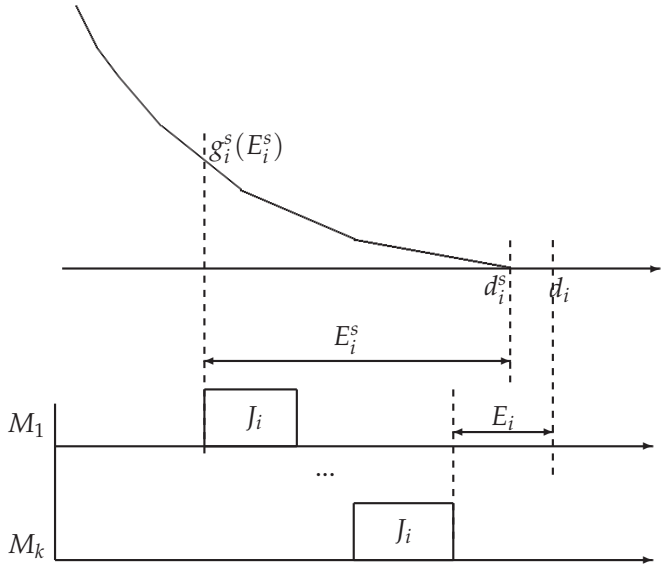

FIG. 1.3 – Calcul de l'avance en fonction de date de début souhaitée

souhaitée $T_i = \max(0, C_i - d_i)$ (voir Figure 1.4).

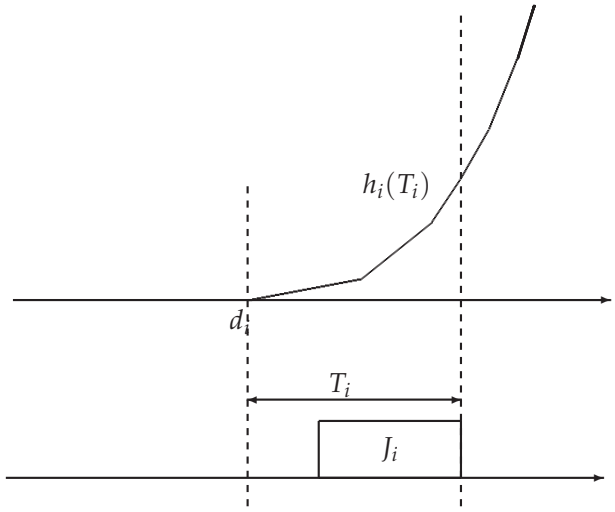

FIG. 1.4 – Calcul du retard

1.3 Mesurer le coût total d'avances et retards des travaux

Selon les motivations et les natures des problèmes, les chercheurs ont défini plusieurs métriques calculant le coût total engendré par l'avance/retard des travaux. En général, la fonction objectif se présente sous la forme d'une combinaison linéaire de

l'avance et retard pondérés. Elle est définie par : $f_i(C_i) = F_i(E_i, T_i) = \sum g_i(E_i) + h_i(T_i)$ où g_i et h_i peuvent être des fonctions en escalier, linéaires, linéaires par morceaux ou simplement convexes. Dans [200, 91], les auteurs considèrent le cas où chaque travail J_i a sa propre fonction de coût, notée $f_i(C_i)$ supposée convexe et linéaire par morceau. C'est une généralisation du cas classique où g_i et h_i sont deux fonctions linéaires. On peut citer d'autres travaux qui définissent la fonction objectif comme étant la somme des valeurs maximales de l'avance et du retard : $E_{\max} + T_{\max}$ et $\max_{1 \leq i \leq n} w_i(E_i + T_i)$ [156].

Dans cette thèse, on s'intéressera en particulier au cas où la fonction de coût d'avance/retard à minimiser est linéaire et continue (voir Figure 1.5).

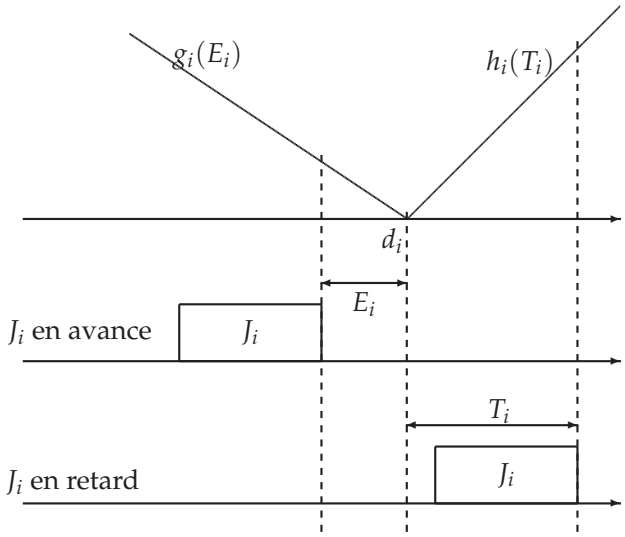

FIG. 1.5 – Fonction de coût d'avance/retard linéaire et continue

Dans, les paragraphes suivants, nous allons classer les différents problèmes d'ordonnancement JàT selon les valeurs des poids associés à l'avance/retard d'un travail.

1.3.1 Fonction de coût linéaire continue

On s'intéresse à minimiser la somme pondérée de l'avance/retard. Supposons que les fonctions de coûts g et h sont linéaires. Ainsi, le coût de l'avance et de retard total est donné par la fonction de coût $F_i(E_i, T_i) = \sum(\alpha_i E_i + \beta_i T_i)$ (voir Figure 1.5).

Selon les valeurs des poids α_i et β_i, quatre problèmes peuvent être identifiés.

1. **Cas unitaire** : si $\alpha_i = \beta_i = w$, $\forall i$; la fonction objectif est donnée par : $F_i(E_i, T_i) = \sum(E_i + T_i)$,

2. **Cas identique** : si $\alpha_i = \alpha$ et $\beta_i = \beta$, $\forall i$; la fonction objectif est donnée par : $F_i(E_i, T_i) = \sum(\alpha E_i + \beta T_i)$,
3. **Cas symétrique** : si $\alpha_i = \beta_i = w_i$, $\forall i$; la fonction objectif est donnée par : $F_i(E_i, T_i) = \sum(w_i(E_i + T_i))$
4. **Cas quelconque** : si α_i et β_i sont quelconques ; la fonction objectif est donnée par : $F_i(E_i, T_i) = \sum(\alpha_i E_i + \beta_i T_i)$

Contrairement aux fonctions objectif usuelles en ordonnancement, les critères considérés ne sont pas réguliers, c'est-à-dire qu'il arrive qu'on puisse améliorer une solution en introduisant un temps mort entre deux travaux consécutifs (voir Figure 1.6 avec deux travaux J_1 et J_2). Définir une séquence de travaux n'est donc pas suffisant pour déterminer une solution optimale, car nous avons également besoin des dates de début d'exécution de chaque travail. Selon la littérature, ce problème est connu sous le nom de "*optimal timing problem*" [41, 200, 92, 90, 189]. Dans le cas d'une seule machine, chercher les meilleures dates de début de traitement des travaux peut être résolu en temps polynomial. Pour les problèmes $\mathcal{NP}$-difficiles, les approches exactes classiques peuvent être utilisées, citons par exemple la programmation linéaire [70], la procédure par séparation et évaluation [59, 223, 134, 100, 157, 182, 64, 202]. Pour les méthodes approchées telles que les règles de séquencement, les algorithmes basés sur la recherche par faisceau filtré (beam search) [173, 62], les recherches par voisinage [228, 91, 201] sont aussi proposées. Pour la rechercher du placement optimal des travaux pour une séquence donnée, nous recommandons au lecteur intéressé les références suivantes [41, 200, 92, 24].

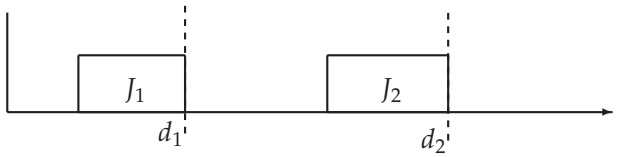

FIG. 1.6 – Juste-à-temps : critère non-régulièr

1.4 Dates de fin souhaitée

Comme les dates de fin souhaitées associées aux travaux sont de plusieurs types, les problèmes d'ordonnancement peuvent être classés en différentes catégories. En géné-

ral, les dates de fin souhaitées sont connues, mais lorsqu'il s'agit de la minimisation de l'avance/retard, ces dates de fin peuvent être variables et à déterminer (cf. par exemple [191, 169, 217]). Dans ce dernier cas, nous parlons de *dates de fin inconnues*. Selon la notation classique en trois champs, nous ajoutons dans le champ β les termes "d_i *unknown*". Dans le cas contraire, ces dates de fin sont des données du problème. Ainsi, si chaque travail a sa propre date de fin, nous parlons de *dates de fin souhaitée arbitraires*.

1.4.1 Date de fin commune donnée

Pour minimiser l'avance/retard, le plus classique est le modèle pour lequel tous les travaux ont une date de fin souhaitée commune. Un tel modèle correspond, par exemple, à un système d'assemblage où les composants d'un produit devraient être prêts en même temps, ou bien lorsqu'il s'agit d'une commande de client qui doit être livrée à une date prévue à l'avance. Plus généralement, ce modèle correspond à tout système où plusieurs travaux devraient être achevés en même temps. On parle d'un problème JàT avec date de fin souhaitée commune, et on note "$d_i = d$" – présenté par [129], la première référence introduisant le problème JàT avec date de fin souhaitée commune. Cependant, on distingue deux types de problèmes :

1. Date de fin commune *non-restrictive* (dite aussi *large*) : si les travaux peuvent être débutés avant la date zéro, i.e. augmenter la valeur d ne permet pas de réduire la valeur de la fonction objectif. Dans le champ β, on ajoute le terme *unrestrictive*.
2. Date de fin commune *restrictive* : dans ce cas on cherche à minimiser l'avance/retard par rapport à une date donnée fixée, i.e. les travaux débutent au plus tôt à la date zéro.

Pour le cas non-restrictive, une structure intéressante définissant une solution optimale a été élaborée dans [85, 61] : il existe une solution optimale où les travaux sont ordonnancés sans temps mort intermédiaire avec l'existence d'un travail k dit juste-à-temps, i.e. $C_k = d$. Généralement, la solution admet une structure "en V" (*V-shape*). Par exemple, le problème $1|d_i = d, unrestrictive|\sum(E_i + T_i)$ admet une solution optimale où les travaux en avance sont ordonnancés selon la règle LPT (*Longest Processing Time First*) et les travaux en retard respectent l'ordre SPT (*Shortest Processing Time First*) [129]. Un algorithme en $O(n^2)$ permet de déterminer l'ensemble des travaux en avance et ceux en retard. Cet algorithme peut être généralisé au cas de m machines parallèles

identiques pour le problème JàT identique (i.e. $\sum(\alpha E_i + \beta T_i)$) [101]. Dans le cas général, le problème a été montré $\mathcal{N}P$-difficile au sens ordinaire [87].

1.4.2 Famille de dates de fin commune donnée

Dans certaines situations, l'atelier de production doit honorer un certain nombre de commandes k. Ainsi, plusieurs produits faisant partie de la même commande, doivent être disponibles en même temps à une date connue fixée par le client ou par une société de transport. Dans ce cas de figure, on se retrouve face à un problème de JàT avec un ensemble de k dates de fin [55, 109, 111]. On note alors dans le champ β : "$d_i \in D$, $|D| = k$". Si k est connu on ajoute alors le terme, "k fixed".

1.4.3 Dates de fin commune contrôlable

Dans certaines situations, la date de livraison est une date à négocier auprès du client et donc elle n'est pas connue à l'avance, c'est donc une variable de décision. On parle d'un problème d'ordonnancement avec affectation de dates de fin. Dans le cas où tous les travaux doivent avoir la même date de fin, optimiser le cycle de production des travaux par exemple, le problème est dit *CON* ("*constant flow allowance*"). Dans la littérature de l'ordonnancement, nous pouvons recenser plusieurs manières dont les dates de fin souhaitées ont été définies. On peut donc donner une classification des problèmes d'ordonnancement avec dates de fins contrôlables comme suit.

1.4.3.1 Modèle SLK

Dans le modèle affectation des dates de fin de type *SLK* ("*SLacK*"), la date de fin souhaitée d'un travail J_i est une fonction de sa date d'arrivée r_i, sa durée opératoire p_i et d'un temps d'attente fixe et commun à tous les travaux noté q. Ainsi, nous avons $d_i = r_i + p_i + q$. Quand tous les travaux ont la même date d'arrivée, nous avons donc $d_i = p_i + q$. Le but est alors de minimiser l'avance/retard des travaux tout en minimisant le temps d'attente commun q.

1.4.3.2 Modèle TWK

Dans le modèle *TWK* ("*Total WorK*"), les dates de fin souhaitées sont des multiples des durées opératoires, données par $d_i = k \times p_i$ avec k un coefficient commun à tous

les travaux. On remarque que pour deux travaux de même durée opératoire, leur date de fin souhaitée est identique. Si les dates d'arrivées ne sont pas nulles, on aura alors $d_i = r_i + k \times p_i$. L'objectif est donc de minimiser l'avance/retard des travaux tout en déterminant la bonne valeur du coefficient k.

1.4.3.3 Modèle PPW

Le modèle *PPW* ("*Processing Plus Wait*") est un modèle combinant les deux règles calculées précédentes : SLK et TWK. Les dates de fin souhaitées des travaux disponibles sont déterminées par $d_i = k \times p_i + q$; sinon par $d_i = r_i + k \times p_i + q$ lorsque les dates d'arrivées ne sont pas identiques. L'affectation des dates de fin souhaitées optimales consiste à déterminer les valeurs optimales de k et q.

1.4.3.4 Modèle NOP

Dans le modèle NOP ("*Number of OPerations*"), les dates de fin souhaitées sont calculées par rapport au nombre d'opérations n_i à exécuter pour la réalisation du travail $J_i : d_i = r_i + k \times n_i$. L'objectif, dans ce cas, est de trouver la valeur optimale de k qui minimise la somme de l'avance/retard des travaux.

1.4.3.5 Autres modèles

D'autres modèles de calcul de dates de fin souhaitées contrôlables ont été considérés dans la littérature de l'ordonnancement. On peut citer par exemple le modèle RDM ("*RanDom Allowance*") où la date de fin souhaitée de chaque travail est générée aléatoirement soit selon une loi de distribution de probabilité ou bien par les méthodes dites de JIQ ("*Jobs In Queue*") et de JIS ("*Jobs In System*"). Selon les deux dernières méthodes, les dates de fin souhaitées sont calculées à partir de la longueur de la file d'attente et du nombre des travaux dans le système. Nous recommandons au lecteur intéressé les références suivantes [183, 213, 214, 73, 194, 151, 185, 176, 210], où l'approche de résolution utilisée est basée sur la simulation.

1.4.4 Dates de fin souhaitées généralisées

Dans certains systèmes de production, le transport des produits finis est négocié auprès des sociétés de transport. Le chef d'atelier dispose donc de plages horaires, autrement dit d'un ensemble de dates de fin souhaitées. Le but est de pouvoir réaliser

Problème	Signification	Date de fin souhaitée
CON	Constant flow allowance	$d_i = d$
SLK	Slack	$d_i = p_i + q$
TWK	Total work	$d_i = k \times p_i$
PPW	Processing plus wait	$d_i = k \times p_i + q$
NOP	Number of operations	$d_i = k \times n_i$
q and k sont les coefficients		
n_i est le nombre des opérations de travail J_i		

TAB. 1.1 – Classification des problèmes d'ordonnancement à affecter date de fin [126]

un ensemble de produits constituant une commande d'un client aux moments clés qui coïncident avec les départs des transporteurs. On parle alors de problèmes d'ordonnancement avec dates de fin souhaitées généralisées ([84, 88]) : l'ensemble des dates de fin souhaitées sont données et le problème consiste à affecter les travaux aux dates de fin.

Une autre variante du cas de date de fin généralisée est abordée dans [102]. Les auteurs parlent de dates de fin souhaitées positionnelles. Quand un travail J_i doit être exécuté pour la $j$ème date de fin souhaitée, on parle de dates de fin souhaitée positionnelles. Par exemple, si on donne trois dates de fin souhaitées d_1, d_2 et d_3 où $d_1 < d_2 < d_3$, pour trois travaux, l'objectif serait de réaliser au moins un travail pour la date d_1, au moins deux travaux pour la date d_2 et tous les autres travaux pour d_3. Cette méthode d'affectation de date de fin permet de savoir le nombre de travaux achevés à des moments clés définis pas les dates de fin souhaitées.

La plupart des problèmes d'ordonnancement sont montrés $\mathcal{NP}$-difficiles. Un point de vue pour résoudre efficacement ces problèmes est d'envisager des méthodes approchées. L'idée sous-jacente est qu'une bonne solution, éventuellement non-optimale mais obtenue en temps raisonnable vaut parfois mieux qu'une solution optimale nécessitant un temps de calcul très important. L'idée de gagner en temps de calcul (ou de rendre le calcul possible) au prix de la qualité de la solution peut alors séduire, voire s'imposer. Les méthodes approchées ne sont envisagées que pour des problèmes d'optimisation pour lesquels on considère qu'une solution satisfaisant les contraintes peut être exploitée sachant qu'elle le sera d'autant mieux si nous pouvons juger sa qualité indépendamment des instances du problèmes considérés, c-à-d. garantir la performance de la méthode proposée. Là encore, cette issue n'est envisageable que si le problème s'y

prête.

Dans la suite de ce chapitre, nous présentons quelques définitions et propriétés connues permettant de classer les problèmes d'ordonnancement du point de vue de leur complexité et des méthodes de résolution.

1.5 Complexité et approximation polynomiale

1.5.1 Approximation et garanties de performances

Pour présenter le principe de l'approximation polynomiale, nous nous baserons sur les travaux de Demange et Paschos [53] où les auteurs font un tour d'horizon de quelques méthodes. Les méthodes approchées diffèrent les une des autres par les compromis qualité/complexité qu'elles offrent. Nous en distinguons deux classes : les heuristiques et les algorithmes polynomiaux à garanties de performances .

Les premières, très utilisées en pratique, n'offrent aucune garantie a priori. Il s'agit de stratégies, de types très variés, dont le principe repose sur des arguments intuitifs, voire sur des paradigmes de phénomènes naturels mais qui ne sont justifiées que par les résultats qu'elles offrent. Aussi, le recours à des plans d'expérience est souvent retenu pour tester ces heuristiques. Dans certains cas, ces méthodes donnent lieu à des analyses complétant les observations ; elles sont systématiques (et même imposées dans les définitions) dans le cadre de l'approximation polynomiale. En effet ce domaine se définit par des règles strictes que doivent satisfaire les algorithmes étudiés. Le principe est de se limiter à l'emploi de méthodes polynomiales en exigeant de plus des garanties absolues sur la qualité des solutions obtenues. Le choix quant aux garanties de complexité et de qualité des solutions obtenues distingue l'approximation polynomiale des autres méthodes approchées. Il est particulièrement intéressant d'étudier le comportement des problèmes difficiles par rapport au compromis complexité/qualité de l'algorithme. L'*approximation polynomiale* restreint son champ d'étude à certains problèmes pouvant s'exprimer en terme d'optimisation. Pour chaque problème d'optimisation, on peut lui faire correspondre un problème de décision exprimés sous forme d'une question à réponse "oui" ou "non" ; la résolution du problème consiste alors à apporter une réponse à la question. À chaque instance, on associe une question portant sur l'existence ou non d'une solution meilleure qu'un seuil fixé. C'est par ce biais qu'on peut faire le lien entre les *problèmes d'optimisation* et les problèmes de décision de la classe

$\mathcal{N}P$. Ainsi, l'ensemble des problèmes d'optimisation $\mathcal{N}P$ est noté $\mathcal{N}PO$. Un problème de décision ne peut pas être plus dur que le problème d'optimisation $\mathcal{N}PO$ correspondant. Lorsque sa version décision est $\mathcal{N}P$-complet , le problème $\mathcal{N}PO$, au moins aussi difficile qu'un problème $\mathcal{N}P$-complet, est dit $\mathcal{N}P$-difficile . Le principal intérêt d'exprimer un problème en termes d'optimisation est d'avoir à sa disposition, non seulement une notion de solution acceptable (ou réalisable) du problème, c'est-à-dire qu'elle exprime par le biais de contraintes ce qu'on désire imposer à n'importe quelle solution envisageable, mais aussi une solution de qualité d'une solution réalisable par référence à la valeur de l'objectif. Selon Paschos [170], *"l'approximation polynomiale, c'est l'art d'obtenir des solutions réalisables, de valeur proche (dans un sens prédéfini) de la valeur optimale et tout cela en temps polynomial"* [170].

En nous plaçant dans le cadre des problèmes difficiles à résoudre, face aux grandes instances l'éventualité de détériorer la valeur optimale pour accélérer le temps de calcul semble être un choix absolument justifié. Cependant, il serait intéressant de pouvoir associer à chaque heuristique une tolérance quant à l'erreur qu'elles peuvent impliquer. C'est ainsi que fut introduite la notion d'erreur relative, c'est-à-dire l'écart maximal par rapport à la valeur optimale de la fonction objectif qu'un algorithme peut produire, et cela quelle que soit l'instance du problème étudié. Plusieurs notions d'approximations ont été présentées dans la littérature (voir [71, 172, 130, 13, 212, 171]), mais nous nous en tiendrons à celle qui est la plus répandue. C'est ainsi que nous obtenons la définition dans le cadre général de l'optimisation :

Définition 1.5.1 *Soit Π un problème de $\mathcal{N}PO$ et un algorithme A pour résoudre ce problème. Étant donnée une instance I du problème et une solution réalisable $f(A(I))$ de I, on définit :*

1. *"l'erreur relative de A respectant I", le rapport*

$$Err(I, A) = \frac{|f^*(I) - f(A(I))|}{\max\{f^*(I), f(A(I))\}}$$

2. *"le rapport de performance de A respectant I", le rapport*

$$R(I, A) = \min\{\frac{f(A(I))}{f^*(I)}, \frac{f^*(I)}{f(A(I))}\}$$

avec $f^*(I)$ la valeur de la solution optimale d'une instance du problème Π.

En nous plaçant dans le cadre des problèmes de minimisation pour les problèmes

d'ordonnancement avec $f^*(I)$ la valeur optimale du problème, $f(A(I))$ la valeur retournée par l'algorithme A et ε l'erreur relative, on obtient les équations

$$f(A(I)) - f^*(I) = \varepsilon f^*(I)$$

soit

$$\frac{f(A(I))}{f^*(I)} = 1 + \varepsilon$$

À partir de ces équations nous pouvons désormais définir la garantie de performance d'une heuristique, c'est-à-dire la valeur de l'écart relatif que l'algorithme A garantit quelle que soit l'instance étudiée.

Définition 1.5.2 *Soit Π un problème de minimisation de $\mathcal{NPO}$. Quelle que soit l'instance I du problème, un algorithme polynomial à garantie de performance A qui retourne une solution réalisable de I, vérifie l'inéquation :*

$$f(A(I)) - f^*(I) \leq \varepsilon f^*(I)$$

soit

$$\frac{f(A(I))}{f^*(I)} \leq 1 + \varepsilon$$

avec ε fixé, $f(A(I))$ le coût de la solution retournée par A et $f^*(I)$ le coût de la solution optimale.

1.5.2 Schémas d'approximation polynomiaux

Dans le cadre de cette thèse, nous nous intéressons particulièrement à l'élaboration de schémas d'approximation pour les problèmes d'ordonnancement étudiés. Précisons tout d'abord ce qu'est un schéma d'approximation. Cette présentation sera basée en grande partie sur les travaux de Woeginger [218], Demange et Paschos [53]. Nous choisissons de présenter la classe des schémas d'approximation de façon formelle. Nous nous plaçons dans la classe des problèmes d'optimisation $\mathcal{NP}$, soit la classe $\mathcal{NPO}$. Afin de présenter cette classe de façon concise, nous allons donner quelques définitions rencontrées dans la littérature [53, 71].

• $\mathcal{NPO}$

Définition 1.5.3 *Soit Π un problème $\mathcal{NPO}$ et A un algorithme qui, pour toute instance I de Π, retourne une solution réalisable $A(I)$. Étant donné un rationnel arbitraire ε, on dit que A est un algorithme d'ε-approximation pour Π si, pour chaque instance I, la solution réalisable $A(I)$ vérifie l'inégalité $\dfrac{f(A(I))}{f^*(I)} \leq 1 + \varepsilon$.*

• $\mathcal{APX}$

Définition 1.5.4 *Un problème Π de $\mathcal{NPO}$ appartient à la classe $\mathcal{APX}$ si un algorithme d'ε-approximation à temps polynomial A pour Π existe.*

Pour synthétiser les définitions précédentes, nous ajoutons qu'un algorithme d'ε-approximation peut être assimilé à une heuristique avec une garantie de performance de $1 + \varepsilon$.

Un problème appartient à la classe $\mathcal{APX}$ si on sait qu'il existe un ε compris entre 0 et 1 satisfaisant cette condition.

Définition 1.5.5 *Soit Π un problème $\mathcal{NPO}$, un algorithme A est un schéma d'approximation pour Π si, pour chaque instance I de Π et pour une famille de rationnel ε, $A(x, \varepsilon)$ renvoie une solution réalisable dont l'erreur relative est au plus ε.*

Un schéma d'approximation pour un problème est donc un algorithme prenant comme paramètre ε. Il est ainsi possible de préciser la garantie de performance de l'algorithme en question.

• $\mathcal{PTAS}$

Définition 1.5.6 *Un problème Π de $\mathcal{NPO}$ appartient à la classe **PTAS** s'il admet un schéma d'approximation à temps polynomial, c'est-à-dire un schéma d'approximation dont la complexité est bornée par $q_\varepsilon(|I|)$, où q_ε est un polynôme.*

Un **PTAS** (*Polynomial Time Approximation Scheme*) est un schéma d'approximation, donc un algorithme, dont la complexité est polynomiale par rapport à la taille de l'instance, même si le polynôme majorant la complexité est en fonction de ε.

• $\mathcal{FPTAS}$

Définition 1.5.7 *Un problème* Π *de* $\mathcal{NPO}$ *appartient à la classe* ***FPTAS*** *s'il admet un schéma d'approximation à temps complètement polynomial, c'est-à-dire un algorithme dont la complexité est bornée par* $q(|I|, 1/\varepsilon)$*, où* q *est un polynôme.*

Un **FPTAS** (*Fully Polynomial Time Approximation Scheme*) est un schéma d'approximation à temps complètement polynomial c'est-à-dire un algorithme dont la complexité est polynomiale en fonction de la taille de l'instance et de l'inverse du paramètre ε. En général, il est possible de déduire un FPTAS à partir d'un algorithme pseudo-polynomial, tel qu'un algorithme de programmation dynamique, quelle que soit l'erreur relative ε choisie.

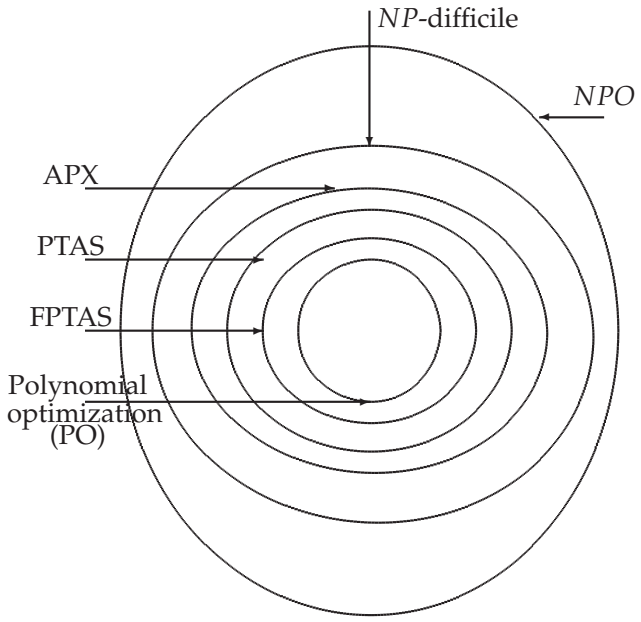

FIG. 1.7 – Classes d'approximabilité (sous l'hypothèse $\mathcal{P} \neq \mathcal{NP}$) [171]

En effet, il est à noter que les problèmes $\mathcal{N}P$-difficiles au sens fort n'admettent pas de schéma d'approximation FPTAS et que certains problèmes $\mathcal{N}P$-difficiles au sens ordinaire n'en admettent pas non plus. Les différentes classes de problèmes présentées s'encapsulent tel que montré sur la figure 1.7. En 1978, Garey et Johnson [71] avaient identifié certaines conditions sur les problèmes d'optimisation qui garantissent que le problème ne possède pas de schéma d'approximation, sous l'hypothèse $\mathcal{P} \neq \mathcal{N}P$. On peut également faire référence à de nombreux autres travaux [172, 15, 14]. Le principe des schémas d'approximation est apparu au milieu des années 70 avec les travaux de Horowitz, Ibarra, Kim, et Sahni [104, 124, 190] et ils sont, pour la plupart, basés sur des formulations de programmation dynamique qui sont, rappelons le, des méthodes

exactes de résolution. On dénote deux grandes familles d'approches pour transformer une formulation de programmation dynamique en schéma d'approximation. La première consiste à arrondir les données d'entrée d'une instance. Le but est de simplifier les données de l'instance (les entrées), ainsi de rendre le problème obtenu facile à résoudre. Nous appelons cette méthode la technique des "entrées arrondies" (*rounding-the-input-data*) [190]. La deuxième méthode consiste à simplifier l'espace de recherche de la programmation dynamique en réduisant l'espace de recherche sur lequel la méthode pseudo-polynomiale doit être appelée (*trimming-the-state-space*) [124].

1.6 Problèmes d'ordonnancement Juste-à-temps abordés

Pour conclure de chapitre introductif, nous avons souligné que l'approche de type Juste-à-Temps peut être déployée aux différents niveaux décisionnels d'un système de production. Dans cette première partie de thèse, nous nous sommes intéressés au niveau opérationnel où nous essayons d'apporter quelques améliorations à l'ordonnancement à court terme dans un atelier de production. Nous nous sommes particulièrement intéressés aux problèmes d'ordonnancement présentés dans le tableau 1.2, où notre but est de proposer des méthodes exactes ou approchées avec garantie de performances pour déterminer une meilleure politique d'ordonnancement des travaux à réaliser.

	Critère	$d_i = d$			$d_i \in D$, $\lvert D \rvert = k$, k fixé
		d donnée		d inconnue	
		non-restrictive	restrictive		
Chapitre 2	$\sum w_i T_i$	-	1 ;Pm ;Qm	-	Pm ;Qm
Chapitre 3	$\sum(\alpha_i E_i + \beta_i T_i)$	-	1 ;Pm ;Qm	-	Pm ;Qm
Chapitre 4	$\sum(\alpha_i E_i + \beta_i T_i + \gamma d)$	-	1 ;Pm ;Qm	-	1,Pm

1 - Une seule machine
Pm - m machines parallèles identiques avec m fixé
Qm - m machines parallèles uniformes avec m fixé

TAB. 1.2 – Tableau de problèmes abordés